

Name: Javaughn Stevens

CMSC 204 UserAccessManager Project 1 Writeup

1. What did you learn?

In this project, I learned about using various exception types to handle certain real world scenarios and why they exist. I also learned about hashed code and how it can be encrypted and decrypted for advanced security, and how JUnit can throw and use exceptions in test cases for thorough testing of complex programs.

2. What issues did you encounter, if any?

I did not encounter any issues when working on this project due to having set up Eclipse IDE, Java JDK 25, JavaFX, and JUnit 5 successfully in the beginning of the semester.

3. What would you have done differently?

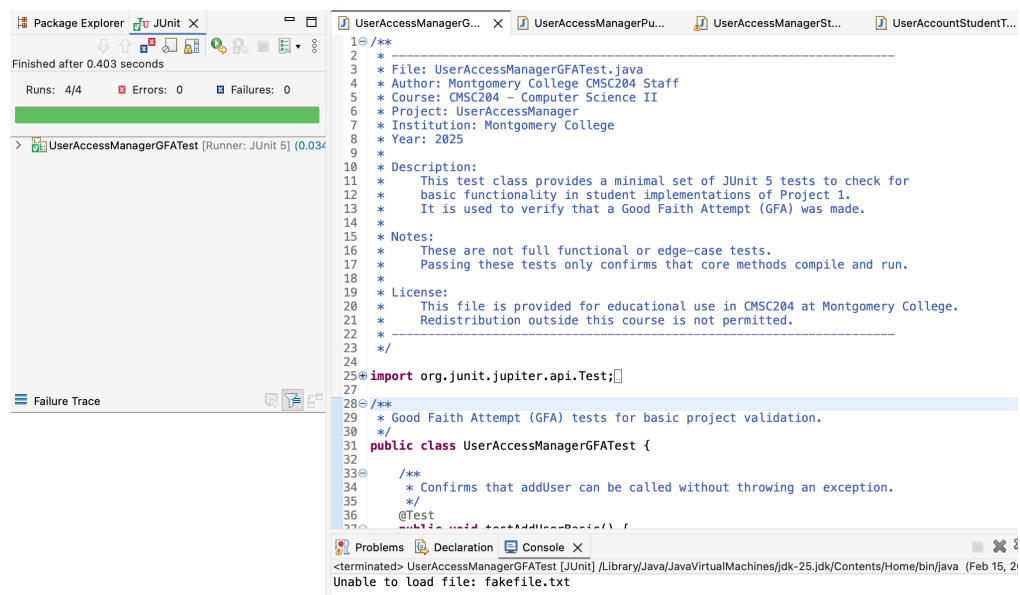
I would not do anything differently since I think my current approach and planning towards completing projects for this course is solid - I typically begin setting up the project near the beginning of the week and review/complete the slides, weekly discussion activity, and lab(s) to gain knowledge to apply to the project implementation. If I had more time, I also would have implemented a way for users to potentially get unlocked if they got locked from their account through other means of identity verification, or a way to change the user's password if they forgot it, since these ideas do exist in many forms of technology we use in our everyday lives (phones, computers, etc.)

4. How can you apply this concept in the future?

This concept is directly applicable to many cybersecurity outlets such as network security, cloud security, data security, firewalls, and more. It could also be implemented into technology that is more advanced like fingerprint recognition or facial recognition, a useful alternative to written/typed passwords that can be forgotten and need to be changed multiple times for technology that is used for extended periods.

JUnit Test Screenshots:

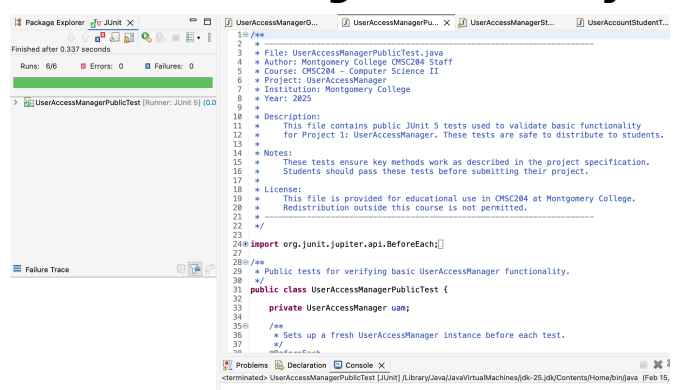
UserAccessManagerGFATest.java:



```
1 /**
2  *
3  * File: UserAccessManagerGFATest.java
4  * Author: Montgomery College CMSC204 Staff
5  * Course: CMSC204 - Computer Science II
6  * Project: UserAccessManager
7  * Institution: Montgomery College
8  * Year: 2025
9  *
10 * Description:
11 * This test class provides a minimal set of JUnit 5 tests to check for
12 * basic functionality in student implementations of Project 1.
13 * It is used to verify that a Good Faith Attempt (GFA) was made.
14 *
15 * Notes:
16 * These are not full functional or edge-case tests.
17 * Passing these tests only confirms that core methods compile and run.
18 *
19 * License:
20 * This file is provided for educational use in CMSC204 at Montgomery College.
21 * Redistribution outside this course is not permitted.
22 */
23
24
25 import org.junit.jupiter.api.Test;
26
27
28 /**
29  * Good Faith Attempt (GFA) tests for basic project validation.
30 */
31 public class UserAccessManagerGFATest {
32
33     /**
34      * Confirms that addUser can be called without throwing an exception.
35      */
36     @Test
37     public void testAddUserBasic() {
38
39     }
40 }
```

terminated> UserAccessManagerGFATest [JUnit] [Library\Java\JavaVirtualMachines\jdk-25.jdk\Contents\Home\bin\java (Feb 15, 2025) 10:00:00 AM] Unable to load file: fakefile.txt

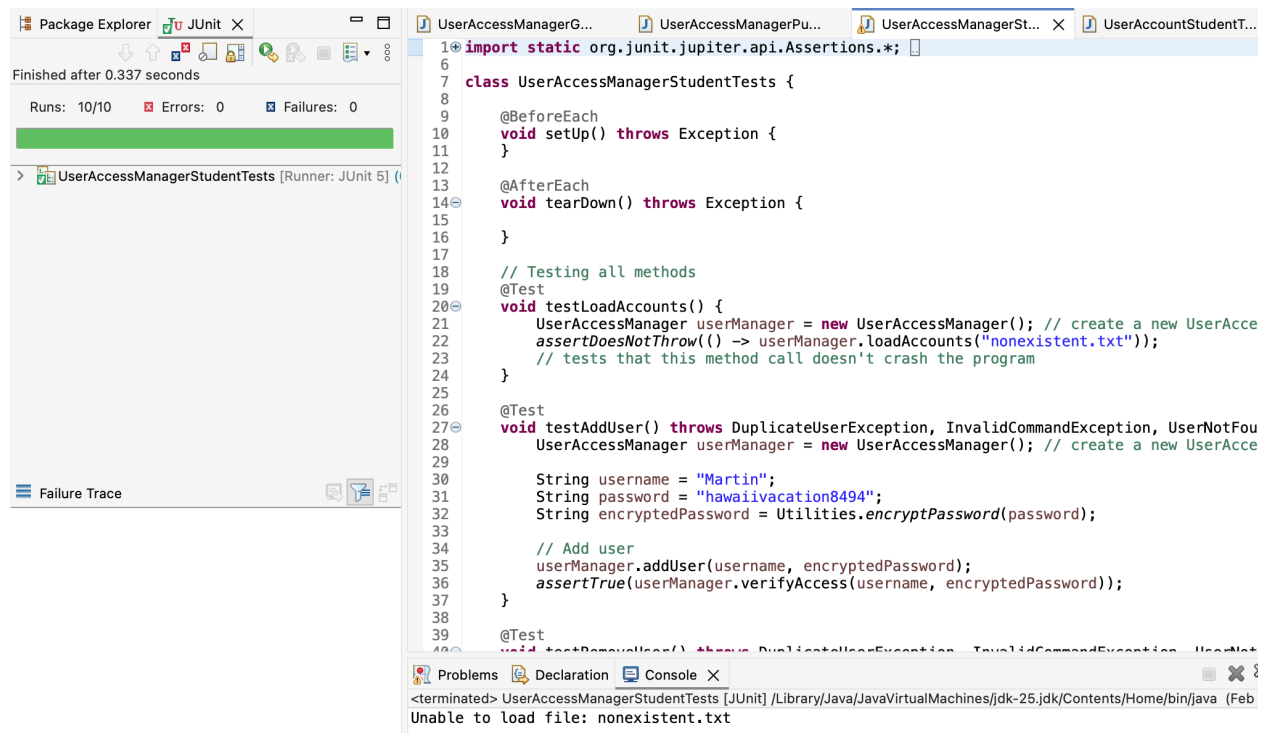
UserAccessManagerPublicTest.java:



```
1 /**
2  *
3  * File: UserAccessManagerPublicTest.java
4  * Author: Montgomery College CMSC204 Staff
5  * Course: CMSC204 - Computer Science II
6  * Project: UserAccessManager
7  * Institution: Montgomery College
8  * Year: 2025
9  *
10 * Description:
11 * This file contains public JUnit 5 tests used to validate basic functionality
12 * for Project 1: UserAccessManager. These tests are safe to distribute to students.
13 *
14 * Notes:
15 * These tests ensure key methods work as described in the project specification.
16 * Students should pass these tests before submitting their project.
17 *
18 * License:
19 * This file is provided for educational use in CMSC204 at Montgomery College.
20 * Redistribution outside this course is not permitted.
21 */
22
23
24 import org.junit.jupiter.api.BeforeEach;
25
26
27
28 /**
29  * Public tests for verifying basic UserAccessManager functionality.
30 */
31 public class UserAccessManagerPublicTest {
32
33     private UserAccessManager uam;
34
35     /**
36      * Sets up a fresh UserAccessManager instance before each test.
37      */
38     @BeforeEach
39     public void setUp() {
40         uam = new UserAccessManager();
41     }
42 }
```

terminated> UserAccessManagerPublicTest [JUnit] [Library\Java\JavaVirtualMachines\jdk-25.jdk\Contents\Home\bin\java (Feb 15, 2025) 10:00:00 AM]

UserAccessManagerStudentTests.java:

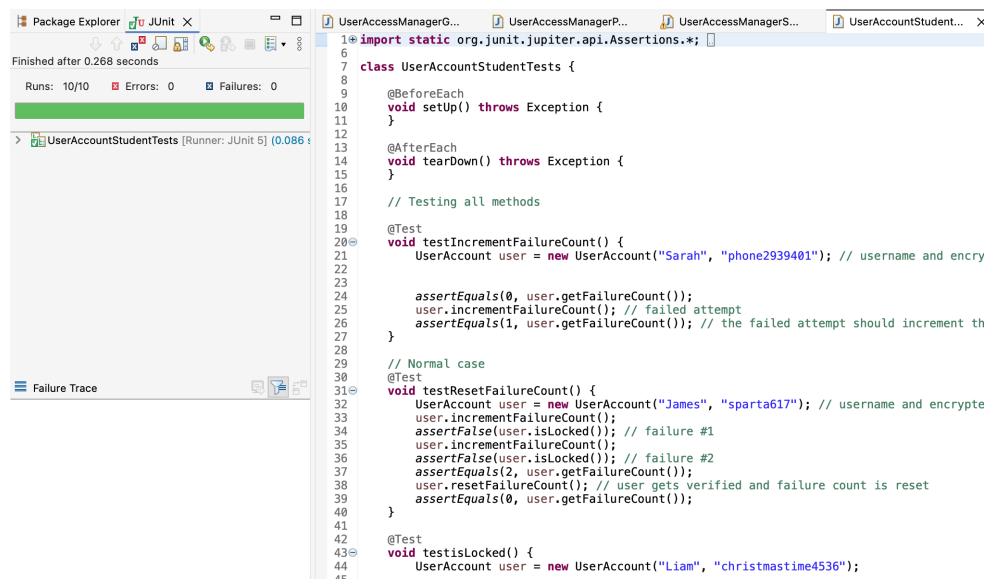


```
1 import static org.junit.jupiter.api.Assertions.*;
6
7 class UserAccessManagerStudentTests {
8
9     @BeforeEach
10    void setUp() throws Exception {
11    }
12
13    @AfterEach
14    void tearDown() throws Exception {
15    }
16
17    // Testing all methods
18    @Test
19    void testLoadAccounts() {
20        UserAccessManager userManager = new UserAccessManager(); // create a new UserAcce
21        assertDoesNotThrow(() -> userManager.loadAccounts("nonexistent.txt"));
22        // tests that this method call doesn't crash the program
23    }
24
25    @Test
26    void testAddUser() throws DuplicateUserException, InvalidCommandException, UserNotFou
27        UserAccessManager userManager = new UserAccessManager(); // create a new UserAcce
28
29        String username = "Martin";
30        String password = "hawaiiivacation8494";
31        String encryptedPassword = Utilities.encryptPassword(password);
32
33        // Add user
34        userManager.addUser(username, encryptedPassword);
35        assertTrue(userManager.verifyAccess(username, encryptedPassword));
36    }
37
38    @Test
39    void testRemoveUser() throws DuplicateUserException, InvalidCommandException, UserNotFou
```

Problems Declaration Console X

<terminated> UserAccessManagerStudentTests [JUnit] /Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java (Feb
Unable to load file: nonexistent.txt

UserAccountStudentTests.java:



```
1 import static org.junit.jupiter.api.Assertions.*;
6
7 class UserAccountStudentTests {
8
9     @BeforeEach
10    void setUp() throws Exception {
11    }
12
13    @AfterEach
14    void tearDown() throws Exception {
15    }
16
17    // Testing all methods
18    @Test
19    void testIncrementFailureCount() {
20        UserAccount user = new UserAccount("Sarah", "phone2939401"); // username and encry
21
22        assertEquals(0, user.getFailureCount());
23        user.incrementFailureCount(); // failed attempt
24        assertEquals(1, user.getFailureCount()); // the failed attempt should increment th
25    }
26
27    // Normal case
28    @Test
29    void testResetFailureCount() {
30        UserAccount user = new UserAccount("James", "sparta617"); // username and encrypt
31        user.incrementFailureCount();
32        assertFalse(user.isLocked()); // failure #1
33        user.incrementFailureCount();
34        assertFalse(user.isLocked()); // failure #2
35        assertEquals(2, user.getFailureCount());
36        user.resetFailureCount(); // user gets verified and failure count is reset
37        assertEquals(0, user.getFailureCount());
38    }
39
40    @Test
41    void testIsLocked() {
42        UserAccount user = new UserAccount("Liam", "christmastime4536");
43
44    }
```

CommandLineIntegrationTests.java is omitted due to the recent Blackboard announcement.